Your LLM Has Been Forgetting Everything — Karpathy's Wiki Pattern Is the Fix

16 min read · Apr 10, 2026

Mustafa Genc Follow

Listen Share More

There's something quietly frustrating about how most people use LLMs for research. You open a session, spend thirty minutes building up context — explaining your topic, your assumptions, what you already know — ask your questions, get your answers, and close the tab. Tomorrow you do it again from scratch. The AI never actually learns anything about your domain. It can't. Every session is stateless.

Andrej Karpathy — former Director of AI at Tesla, co-founder of OpenAI, and the person behind the "Neural Networks: Zero to Hero" course — posted something in early April 2026 that directly addresses this problem. Not with a new model or a product. With a workflow: a pattern for using LLMs to build and maintain a persistent personal knowledge base. He called it "LLM Knowledge Bases," and within days it had 5,000+ stars, 1,900+ forks, and a comment thread full of people shipping implementations.

The core claim is simple but worth examining carefully: instead of querying raw documents at runtime, you have the LLM incrementally compile everything into a structured wiki that compounds over time. The devil is in the details — and so are the real tradeoffs.

> *If you liked this article, please clap — and if you're feeling generous, you can give up to 50 claps* 👏

Why RAG Is Not Enough for Personal Knowledge Work

To understand what Karpathy is proposing, you need to understand what standard RAG actually does — and where it falls short.

Retrieval-Augmented Generation (RAG) is the dominant pattern for letting an LLM answer questions from your own documents. Documents are chunked into fragments, converted into vector embeddings, stored in a database (Pinecone, Weaviate, pgvector, etc.), and at query time the system searches for the most similar chunks and injects them as context. Tools like NotebookLM, ChatGPT file uploads, and most enterprise document search products work this way.

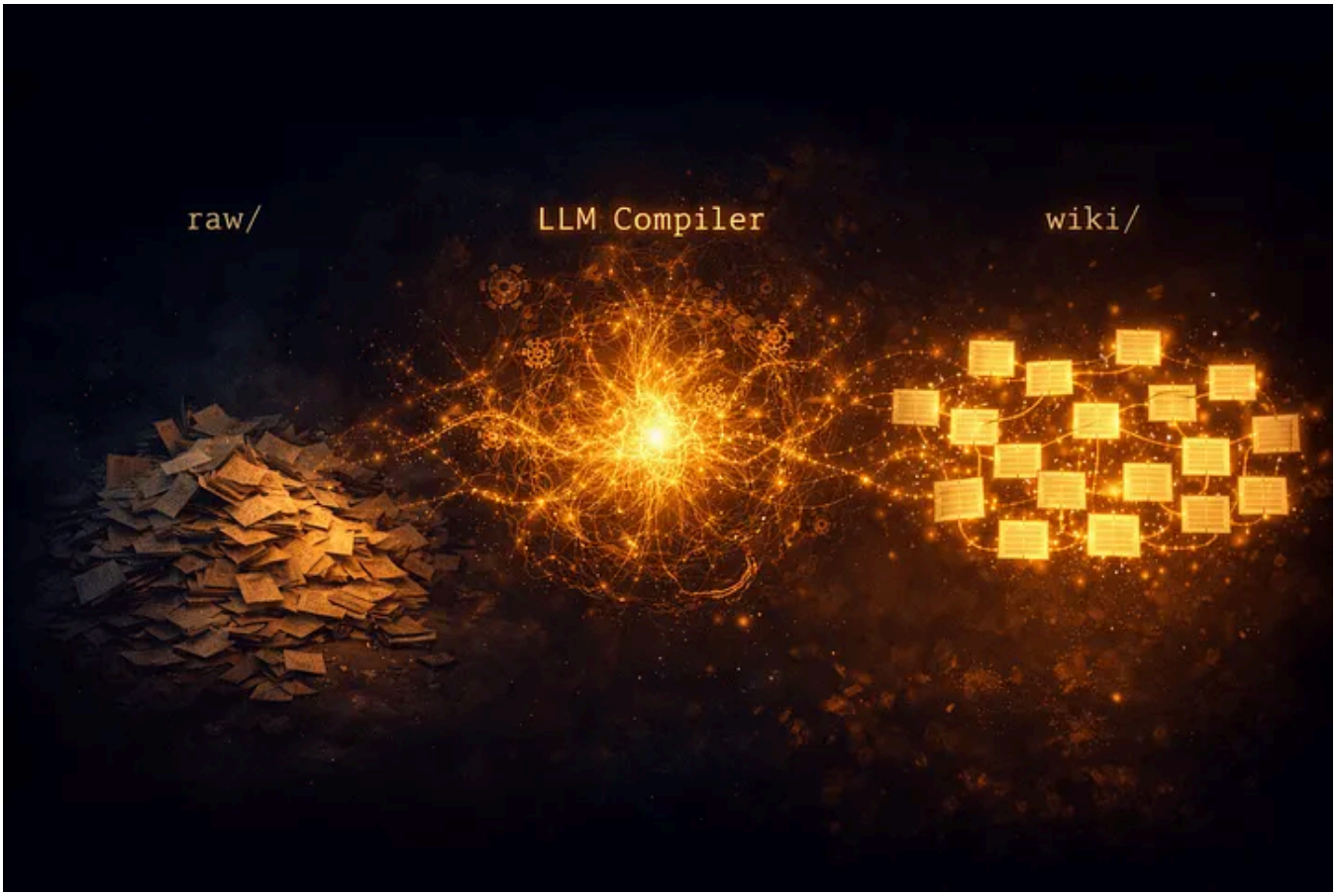
The fundamental problem is that nothing accumulates. Every query starts from scratch. If you ask a question that requires synthesizing five different documents, the system has to find and reassemble all five fragments at runtime — and if the chunking wasn't perfect, or the embedding similarity doesn't surface the right pieces, you get a degraded answer. More subtly: the system learns nothing from the fact that you asked the question. The next query is just as blind as the first.

Karpathy's critique is precise. As he put it in the gist: "The LLM is rediscovering knowledge from scratch on every question. There's no accumulation."

The chunking issue is also real. When documents are split into arbitrary fragments, they lose surrounding context. A paragraph that makes sense in relation to the section before and after it becomes an isolated snippet. The LLM that reads it at

The Compiler Metaphor: How LLM Wiki Actually Works

Karpathy's alternative replaces retrieval with compilation. The LLM doesn't retrieve from raw documents — it reads them, synthesizes them, and writes a structured wiki that persists between sessions. Future queries read from the wiki, not the raw sources.



The architecture has three layers:

Raw sources (`raw/`): This is your collection of immutable source material — papers, articles, GitHub repos, datasets, images. The LLM reads from here but never modifies anything. Think of it as your source of truth.

The wiki (`wiki/`): A directory of LLM-generated markdown files. Summaries, entity pages, concept articles, comparisons, an evolving index. The LLM owns this layer entirely. It creates pages, updates them as new sources arrive, maintains cross-references, and flags contradictions. You read it; the LLM writes it.

The schema (CLAUDE.md / AGENTS.md): A configuration file that tells the LLM how the wiki is structured, what conventions to follow, and what workflows to use when ingesting sources or answering questions. Karpathy notes this file is co-evolved — you refine it over time as you figure out what works for your domain.

When you add a new source, the LLM doesn't just index it. It reads the full document, extracts key information, writes a summary page, and integrates the new content into existing wiki articles — updating entity pages, noting where new data contradicts old claims, strengthening or revising the evolving synthesis. A single source might touch ten to fifteen wiki pages. The knowledge is compiled once and kept current.

The query model changes completely. Instead of a similarity search over raw chunks, the LLM reads the wiki's index file (a compact summary of all pages), identifies the relevant articles, and synthesizes an answer from pre-compiled, human-readable content. No embeddings. No retrieval noise. Karpathy reports running this at roughly 100 articles and 400,000 words — longer than most PhD dissertations — and finding that a modern LLM can navigate it well via the index without needing a full RAG stack.

Here's a minimal schema that illustrates the pattern:

```
# Knowledge Base Schema

## What This Is
A personal knowledge base about [YOUR TOPIC].

## Directory Structure
- raw/ — source documents. Immutable. LLM reads, never writes.
- wiki/ — compiled knowledge. LLM maintains entirely.
- outputs/ — generated answers, reports, visualizations.

## Wiki Conventions
- Every topic gets its own .md file
- Every page starts with a one-paragraph summary (TLDR)
- Use [[topic-name]] for cross-references
```

```
- Maintain INDEX.md: every page listed with a one-line summary
- Maintain LOG.md: append-only record of every ingest and query

## On Ingest
1. Read source, discuss key takeaways
2. Write summary page in wiki/
3. Update INDEX.md
4. Update relevant entity and concept pages
5. Note any contradictions with existing content
6. Append entry to LOG.md
```

And a Claude Code workflow to kick things off:

```
# Point Claude Code at your project root, then:
# "Read everything in raw/. Compile a wiki in wiki/
# following the rules in CLAUDE.md. Create INDEX.md first,
# then one .md file per major topic. Link related topics."
```

The 7 Things That Make It Actually Stick

The three-layer architecture above is the skeleton. But a skeleton doesn't move on its own. What the community distilled — across 120+ comments on Karpathy's original gist — is that most people who try this pattern get the folder structure right and still end up with a wiki that slowly becomes unreliable, redundant, or just abandoned. The difference between a wiki that compounds and one that quietly rots comes down to seven operational practices. None of them are technically complex. All of them are easy to skip.

Source: Shann3

01 — Separate Your Vaults

This one is subtle but critical. The instinct when starting is to dump everything into one place — your own notes, your agent's speculative writes, half-finished ideas, things you're not sure about yet. That instinct will eventually destroy the signal in your wiki.

The principle is simple: your vault (personal thinking, curated notes, verified knowledge) and the agent's working vault (speculative writes, messy drafts, exploratory connections the LLM is still testing) need to be physically separate directories. When an LLM writes speculatively into the same space where your

curated signal lives, it contaminates it. You start questioning whether a page was something you verified or something the agent hallucinated at 2am during an unsupervised batch run. That doubt compounds. Eventually you trust nothing.

Think of it like a compiler's separation between source code and build artifacts. You never manually edit the build output, and the compiler never touches your source. Same principle here.

02 — Classify Before You Extract

Not all documents carry information the same way, and treating them as if they do is one of the most common ways to lose value on ingest.

A research paper leads with its method and findings — the abstract and conclusion often contain 80% of what you need, and the contribution is usually a specific, falsifiable claim. A meeting transcript carries meaning through speaker attribution — who said what matters as much as what was said, because position and authority change the weight of a statement. An executive report should be summarized exec-summary-first, with supporting detail treated as secondary, because the structure itself signals what the author considered most important.

A generic extraction prompt that ignores document type consistently misses what makes each format valuable. Your schema should define type-specific ingest rules — a few lines per document type that tell the LLM what to prioritize, what structure to expect, and what to do with metadata like authors, dates, and citations. The overhead is minimal; the improvement in wiki quality is significant.

03 — Add a Bias Check on Every Update

An LLM without explicit instructions to the contrary will agree with almost everything it reads. It will ingest a paper arguing X, write a confident wiki page summarizing X, then ingest a paper arguing the opposite of X and write an equally confident wiki page. Without a mechanism to flag the contradiction, both pages sit in your wiki presenting opposing claims with equal confidence. You won't notice until you ask a question that depends on knowing which is right.

The fix is structural: every update to the wiki should force the LLM to also populate a **counter-arguments and data gaps** section on any page it writes or revises. Not as an afterthought — as a required field in your schema. "What does the strongest

opposing view say? What does this source not address? What would change this conclusion?"

Without this, the wiki becomes an echo chamber in a tuxedo. It looks authoritative. It agrees with everything you fed it. It is confidently wrong.

04 — Put a TLDR on Every Page

This is as much a performance optimization as it is a usability feature.

When a query arrives, the LLM doesn't load every page in the wiki — that would be prohibitively expensive at any meaningful scale. Instead, it scans `index.md`, which lists every page with a one-line summary. If a page looks relevant, it loads the full content. If not, it skips and moves on.

This means the quality of your TLDRs directly determines how well the LLM navigates your wiki. A vague TLDR ("Notes on machine learning") forces the LLM to load the full page speculatively just to check if it's relevant. A precise TLDR ("Comparison of BM25 vs. vector search for hybrid retrieval at 10K document scale, with latency benchmarks") lets the LLM make a confident routing decision without loading anything.

Enforce TLDRs in your schema. Make them required. Make them specific. They are the index your LLM uses to think.

05 — File Queries Back Into the Wiki

This is the compounding mechanism. Most people miss it entirely.

When you ask a good question and the LLM synthesizes a genuinely useful answer — a comparison, an analysis, a connection between two ideas you hadn't linked before — that answer lives in your chat history and nowhere else. The next session, it's gone. The LLM starts from the same wiki it had before you asked the question. The insight evaporated.

Filing answers back into the wiki as new pages changes this completely. The wiki gets richer with every question you ask. A comparison table you asked for becomes a permanent reference. An analysis you needed once becomes context for the next ten questions. The effort compounds rather than resets.

Karpathy is explicit about this in the gist: outputs should be filed back. In practice, it means adding a step to your query workflow — "if this answer is worth keeping, write it to `wiki/` as a new page and update `index.md`." It takes thirty seconds to instruct. It changes the long-term trajectory of the knowledge base entirely.

06 — Plan for Scale Early

The file-based approach that feels effortless at 50 pages starts to show cracks around 300, and becomes genuinely painful at 500+ if you haven't prepared for it. The problem is not storage — markdown files are tiny. The problem is navigation and search.

The community has converged on a rough three-tier model. At **0–300 pages**, flat files and a well-maintained `index.md` are sufficient. The LLM can navigate by reading the index and drilling into relevant pages. At **300–500 pages**, add proper full-text search — FTS5 (SQLite) or BM25 — before you need it, not after. At **500+ pages**, move to a structured database with full-text search (PostgreSQL is the common choice) and consider frontmatter-based metadata so tools like Obsidian's Dataview plugin can generate dynamic views.

The critical point: the naming conventions and frontmatter structure you establish on day one will determine how painful or painless this migration is. If every page has consistent metadata (date, source count, topic tags, document type), scaling is a query problem. If your pages are inconsistently named and metadata-free, scaling is a reconstruction problem. Make the decision early, document it in your schema, and the LLM will enforce it on every page it writes going forward.

07 — Run Lint Passes Regularly

A wiki that is never audited will slowly diverge from reality. Sources become outdated. Claims made six months ago contradict newer evidence. Pages that were relevant when written become orphans as the wiki evolves around them. Topics get mentioned in five different pages without ever getting their own.

The lint pass is a periodic health check — tell the LLM to scan the entire wiki and look for four specific things:

- **Contradictions**: pages that make claims which directly disagree with each other

- **Stale claims:** statements citing sources that are outdated or have since been superseded by newer evidence (a web search tool helps here)
- **Orphan pages:** pages that exist but nothing links to — usually a sign the wiki's structure has grown around them without integrating them
- **New article candidates:** topics mentioned repeatedly across pages but never given their own page — these are gaps in your coverage you hadn't noticed

The LLM is genuinely good at all four. Running this monthly takes fifteen minutes and catches drift before it becomes structural. Skip it for six months and you'll find yourself manually auditing pages you no longer trust, which defeats the entire purpose of the system.

These seven practices are not optional polish on top of the core pattern. They are what separates a knowledge base that gets smarter over time from one that slowly becomes a liability. The folder structure takes ten minutes to set up. Getting these seven right is the actual work.

The Compounding Effect — and Where It Breaks Down

The strongest argument for this pattern is compounding. Every time you ask a question and file the answer back into the wiki, the next question starts with richer context. Every source you ingest strengthens the cross-references. The wiki becomes a progressively more accurate model of your domain, not just a pile of raw documents.

Karpathy is explicit about this: you should file outputs back into the wiki. A comparison you asked for, an analysis, a connection you discovered — these are valuable and shouldn't disappear into chat history.

This is also where the main failure mode lives.

One commenter on the gist (cited in the thread) pointed out the core risk: when outputs get filed back, errors compound too. If the LLM writes something slightly wrong during a synthesis pass and you save it back as context, future answers build on that wrong thing. Unlike a vector database where the raw source always stays intact, the wiki is a derived artifact — and derived artifacts can drift from reality.

Several community members flagged this directly. The gist thread contains a detailed analysis of three specific failure modes: error accumulation and drift, partial context (the LLM only sees a subset of documents during updates), and information loss through compression (summarization discards nuance you can't recover later).

These are real problems. The mitigations Karpathy describes — periodic lint passes where the LLM audits the wiki for contradictions and unsupported claims — help, but they're not a complete solution. The lint step itself can miss things, and running it periodically means errors can accumulate before they're caught.

The honest framing: this pattern works well for knowledge synthesis where some approximation is acceptable (research, exploration, analysis), and works less well for situations requiring high factual precision or audit trails. For regulatory documents, legal text, or medical information where every claim needs to trace cleanly to a source, a hybrid approach — keeping the raw sources alongside a well-structured wiki — is safer than relying on the compiled layer alone.

The Tooling: Obsidian, the Schema File, and a Search Engine When You Need One

Karpathy's tool choices are minimal on purpose. Obsidian serves as the "IDE frontend" — a markdown viewer where the LLM does the writing and you do the reading. The graph view is particularly useful for seeing which wiki pages are hubs and which are orphans. The Obsidian Web Clipper browser extension converts web articles to markdown for fast ingestion.

For images, there's a useful trick: in Obsidian Settings → Files and links, set the attachment folder to `raw/assets/`. Then bind a hotkey to "Download attachments for current file." After clipping an article, hit the hotkey and all images are saved locally — which lets a vision-capable LLM reference them directly.

The schema file (CLAUDE.md for Claude Code, AGENTS.md for Codex) is the most important single artifact in the system. It's the LLM's instruction manual for your specific knowledge base. Karpathy recommends keeping it "super simple and flat"

— not a configuration object but a readable document that describes the conventions and workflows in plain language.

For search, Karpathy built a simple search engine himself ("vibe-coded a naive one") and later the gist references qmd as a more robust option: a local hybrid BM25/vector search over markdown files with both a CLI interface (so the LLM can shell out to it as a tool) and an MCP server. At 100–200 pages the index file is usually sufficient; beyond that, you want proper search.

One practical gotcha: LLMs can't natively read a markdown file and its embedded images in one pass. The workaround is to have the LLM read the text first, then view referenced images separately. It's a bit clunky but works well enough at this scale.

Where RAG Still Wins, and Where This Pattern Fits

This isn't a general-purpose RAG replacement. The scale boundary matters.

At personal knowledge base scale — hundreds of sources, a few hundred wiki pages — the index-based navigation approach works. Modern LLMs have context windows large enough to hold the index plus several full articles simultaneously. This is context-loading, not retrieval, and it avoids the chunking and retrieval noise problems entirely.

At larger scales — thousands of documents, millions of words — the index itself becomes too large to fit in context, and retrieval becomes necessary again. The wiki pattern is explicitly designed for personal and small-team use.

Karpathy acknowledges a future direction: once the wiki is mature and well-refined, you could generate synthetic training data from it and fine-tune an LLM so it "knows" the domain in its weights, not just in context. That would turn a personal knowledge base into a personalized model — a much more ambitious goal, but a logical extension of the same pattern.

For the right use cases (research deep-dives, competitive analysis, book notes, personal learning logs, internal team wikis on a specific topic), the tradeoffs are favorable. You get human-readable state, traceable claims, and a compounding

knowledge structure that RAG doesn't offer. You give up the scalability and retrieval precision of a well-engineered vector database.

An 80-Year-Old Idea, Finally Executable

Karpathy himself draws the connection to Vannevar Bush's 1945 essay "As We May Think," published in *The Atlantic*. Bush envisioned the Memex — a personal device in which an individual could store all their books, records, and communications, navigable by associative trails rather than rigid categorical indexes. He wrote: "Our ineptitude at getting at the record is largely caused by the artificiality of systems of indexing."

Bush's vision was closer to what Karpathy is describing than to what the web became: private, actively curated, with the connections between documents as valuable as the documents themselves. The part Bush couldn't solve was who does the maintenance. Building and sustaining a personal wiki requires exactly the kind of tedious bookkeeping — updating cross-references, keeping summaries current, noting when new data contradicts old claims — that humans consistently don't do. Personal wikis fail because maintenance costs grow faster than value. People abandon Notion setups for the same reason they abandoned Evernote.

The LLM handles the maintenance. That's the actual unlock.

As Karpathy put it in the gist: "The human's job is to curate sources, direct the analysis, ask good questions, and think about what it all means. The LLM's job is everything else."

Whether that's exactly right depends on how well you write your schema, how carefully you review LLM-generated pages, and how seriously you take the lint step. But the division of labor is sound. You're finally not the bottleneck.

What to Actually Do With This

If you want to try this pattern, the GitHub Gist is the right starting point. It's intentionally abstract — a pattern description, not an implementation — which is

itself a deliberate choice. Karpathy didn't release code. He released an idea file, designed to be pasted into an LLM agent that then instantiates a version customized to your domain and workflow.

The community has moved fast. There are already several open-source implementations worth looking at depending on your stack: agent-wiki (pure markdown, works with Claude Code, Codex, or Cursor), llm-wiki-compiler (npm CLI with incremental rebuilds), and MindOS (multi-agent, shares a single wiki across Claude Code, Cursor, Gemini CLI, and others).

The most important thing to get right is the schema file. The quality of your CLAUDE.md determines how disciplined the LLM will be about wiki maintenance. A vague schema produces vague wiki pages. A schema that clearly defines what a concept article looks like, how to handle contradictions, and when to split a page versus extending an existing one will produce a wiki you can actually rely on.

And run the lint step regularly. The error compounding risk is real. A monthly health check — "find contradictions between pages, flag stale claims, list orphan pages, suggest new articles" — is cheap and catches drift before it becomes structural.

Sources

- Karpathy, A. (2026, April 4). *LLM Wiki* [GitHub Gist]. https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f
- Bush, V. (1945, July). *As We May Think*. The Atlantic. https://www.theatlantic.com/magazine/archive/1945/07/as-we-may-think/303881/
- Buckland, M. (2011). *Still Building the Memex*. Communications of the ACM. https://cacm.acm.org/magazines/2011/2/104378-still-building-the-memex/abstract
- VentureBeat analysis of the LLM Wiki pattern: https://venturebeat.com/data/karpathy-shares-llm-knowledge-base-architecture-that-bypasses-rag-with-an

- TechBuddies deep-dive on architecture tradeoffs: https://www.techbuddies.io/2026/04/04/inside-karpathys-llm-knowledge-base-a-markdown-first-alternative-to-rag-for-autonomous-archives/
- qmd (local markdown search engine): https://github.com/tobi/qmd

Llmwiki | Context Rot | Llm Context Window | Andrej Karpathy | Retrieval Augmented Gen

Follow

Written by Mustafa Genc

164 followers · 44 following

ML Engineer (M.Sc. TUM) skilled in LLMs, NLP, RAG, document processing, clustering, OCR, MLOps, and scalable ML apps with FastAPI & Docker.